Tomasulo's Algorithm Simulator

Basel Fakhri, Husein Amro

May 11, 2025

1 Introduction

This report describes the implementation of a simulator for Tomasulo's algorithm, a dynamic scheduling algorithm used to optimize CPU performance by managing instruction-level parallelism.

2 General Assumptions

The following assumptions are considered in our simulation:

- ALUs, multipliers, dividers, load/store units are pipelined.
- There is one shared reservation station queue between the two ALUs.
- The latency of multiplication is assumed equal to division latency, set at 20 cycles, as they share a single functional unit.
- The simulator consists of the following pipeline stages:
  1. **Issue**: The instruction is placed into the appropriate reservation station if available; otherwise, it stalls due to a structural hazard.
  2. **Reservation Station**: Instructions stall in the reservation station until:
     (a) The corresponding functional unit becomes available (structural hazard).
     (b) The instruction's source operands become available (data hazard).
  3. **Execute**: Execution proceeds without stalls once the instruction enters this stage.
  4. **Write-Back (WB)**: Only one instruction may write back to the register file each cycle; additional instructions stall in a queue due to structural hazards.
- Write-after-write (WAW) hazards are handled by register renaming in the reservation station using tags.

3 Simulator Implementation Details

Our simulator supports the following instructions: `ADD`, `SUB`, `MUL`, `DIV`, `LOAD`, and `STORE`.

The fixed instruction latencies used in the simulator are:

- `ADD`, `SUB`: 2 cycles
- `MUL`: 10 cycles
- `DIV`: 20 cycles
- `LOAD`, `STORE`: 5 cycles

4 Input and Output Formats

4.1 Input Trace Format

Instructions are provided as plain text traces, one instruction per line, formatted as:

```
 , ,          ; Arithmetic
 , ()       ; LOAD
 (),         ; STORE
```

Example:

```
ADD R1, R2, R3
MUL R4, R1, R5
LOAD R6, 0(R7)
STORE 4(R5), R1
DIV R2, R4, R3
```

4.2 Output Metrics

The simulator provides the following output metrics:

- Total execution time (clock cycles including stalls)
- Average instructions per cycle (IPC)
- Average reservation station occupancy (percentage)
- Load/store buffer utilization (percentage)
- Number and percentage of stall cycles due to structural hazards

5 Conclusion

This simulator effectively demonstrates Tomasulo's dynamic scheduling algorithm, accounting for structural, data, and WAW hazards. The simulation highlights how reservation stations, register renaming, and pipelining effectively mitigate hazards, enhancing CPU performance through instruction-level parallelism.